Unpacking Morphine

Targets: `00\calc_m0.exe` and `01\calc_m1.exe`

Ahmed Mahfooz Ali Khan

1 Goal

Dynamically unpack each Morphine-protected sample, recover the Original Entry Point (OEP), dump the memory image, rebuild the Import Address Table (IAT), fix the dump, and verify correct execution.

2 Overview of Morphine

Morphine encrypts the payload and uses a polymorphic decryptor with junk instructions and stack noise to hinder static analysis. Each build mutates, so stubs differ between runs. The reliable workflow is: trace to OEP → dump at OEP → rebuild IAT → fix dump → run and verify.

3 Lab Setup

- **Tools**: PEiD (packer identification), debugger (to reach OEP), PETools (dump), Import REConstructor (ImpREC) for IAT repair, PEview for inspection.
- **Success Criteria**: (1) OEP found, (2) dumped at OEP, (3) IAT rebuilt, (4) dump fixed, (5) fixed binary runs like Windows Calculator.

4 Sample `00\calc_m0.exe`

4.1 Reaching the OEP

After the Morphine stub performs in-memory decryption, control is transferred into the original code via a computed jump (e.g., `jmp eax`). We paused execution at the first instruction of the original code.

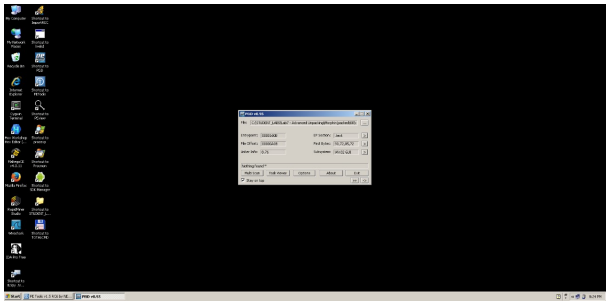

Figure 1: EP detected in the PEID

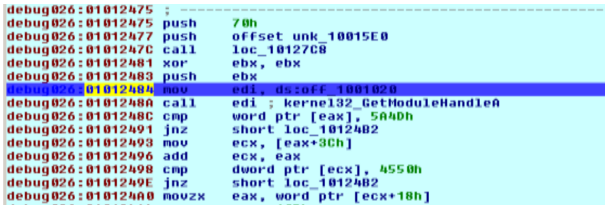

Figure 2: OEP break for sample 00 (`calc_m0.exe`).

4.2 Dump at OEP

PETools was used to dump the process image to disk as `Dumped.exe`.

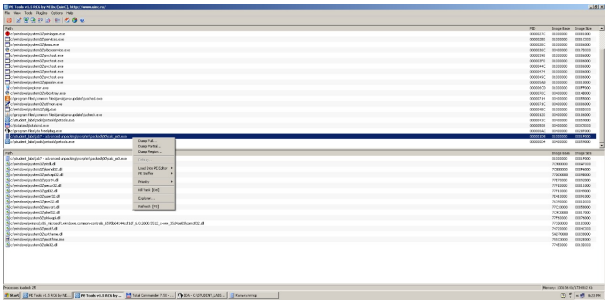

Figure 3: Dumped Full

4.3 Rebuild the IAT (ImpREC)

ImpREC was attached to the running process, the default IAT search range was used, and *Get Imports* followed by *Fix Dump* was executed. The expected imports from `KERNEL32`, `USER32`, `GDI32`, `ADVAPI32`, `SHELL32`, and `MSVCRT` were restored.

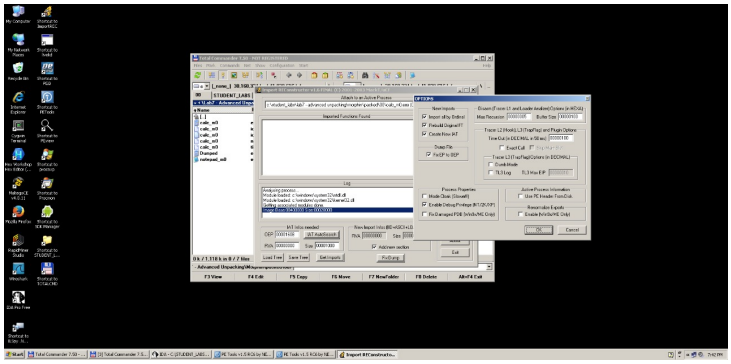

Figure 4: ImpREC detailed view and scan log for sample 00. "New Import Infos" section confirms the rebuilt IAT written into the dump.

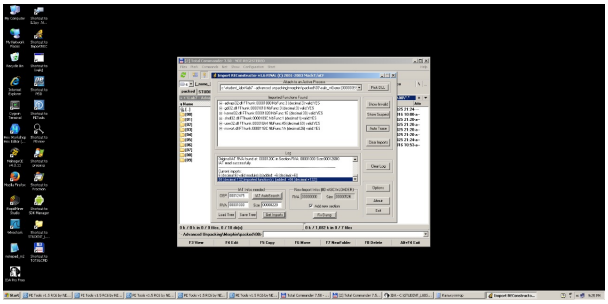

Figure 5: ImpREC module list for sample 00 showing reconstructed imports and valid thunks.

4.4 Fix Dump & Run

ImpREC produced the fixed file `Dumped_.exe`. Launching the file reproduced a working Calculator UI.

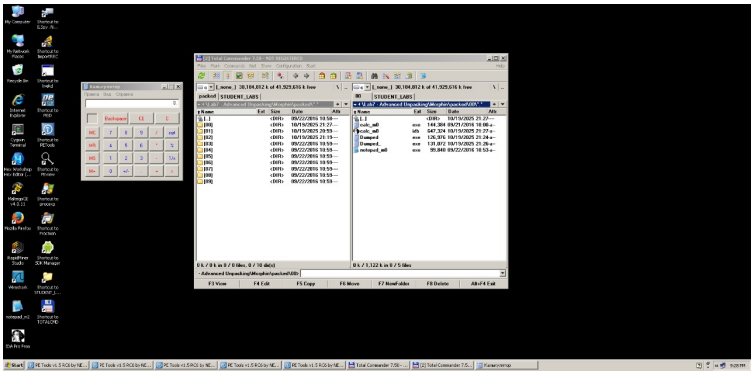

Figure 6: Fixed dump from sample 00 running correctly (Calculator UI visible).

Result (00). OEP captured, process dumped, IAT rebuilt, fixed dump executes correctly.

5 Sample `01\calc_m1.exe`

5.1 Reaching the OEP

This build exhibited a polymorphic variant of the stub (different junk blocks and register noise), but followed the same decrypt-and-jump pattern. The debugger was paused at the OEP.

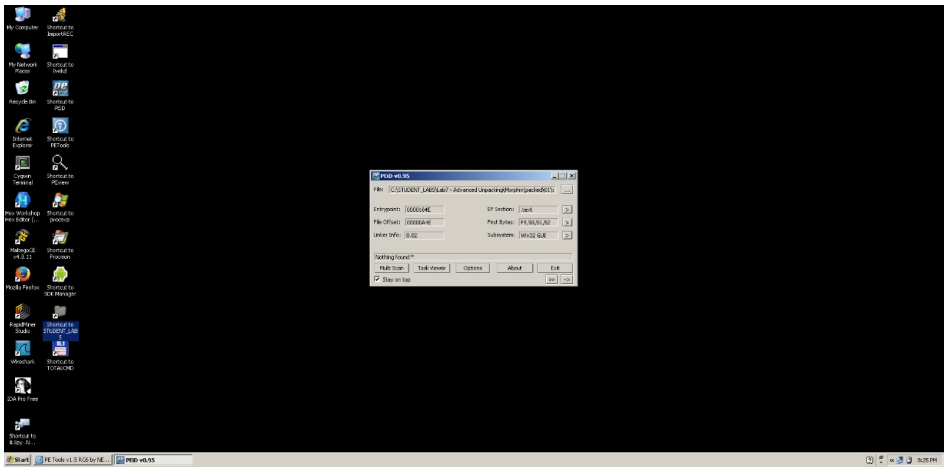

Figure 7: EP detected by PEID (`calc_m1.exe`).

```
debug026:01012475 ; ---------------------------------------------------------------------------
debug026:01012475 push    70h
debug026:01012477 push    offset unk_10015E0
debug026:0101247C call    loc_10127C8
debug026:01012481 xor     ebx, ebx
debug026:01012483 push    ebx
debug026:01012484 mov     edi, ds:off_1001020
debug026:0101248A call    edi ; kernel32_GetModuleHandleA
debug026:0101248C cmp     word ptr [eax], 5A4Dh
debug026:01012491 jnz     short loc_1012482
debug026:01012493 mov     ecx, [eax+3Ch]
debug026:01012496 add     ecx, eax
debug026:01012498 cmp     dword ptr [ecx], 4550h
debug026:0101249E jnz     short loc_1012482
debug026:010124A0 movzx   eax, word ptr [ecx+18h]
```

Figure 8: OEP break for sample 01 (`calc_m1.exe`).

5.2 Dump at OEP

As with sample 01, PETools was used to dumped the exe file.

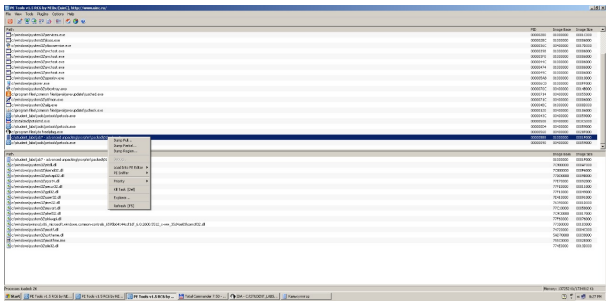

Figure 9: Dumped Full m1 calc

5.3 Rebuild the IAT (ImpREC)

ImpREC scanning showed transient "Can't match RVA ... (Exact call)" messages (common with Morphine's noisy thunks), but converged on a valid descriptor set. *Fix Dump* was then applied.

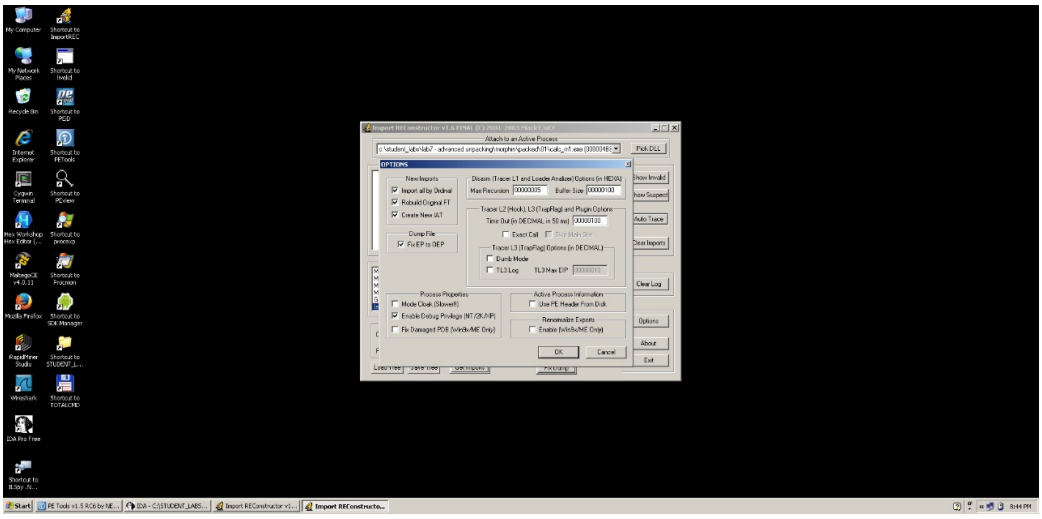

Figure 10: ImpREC detailed view and scan log for sample 01. "New Import Infos" section confirms the rebuilt IAT written into the dump

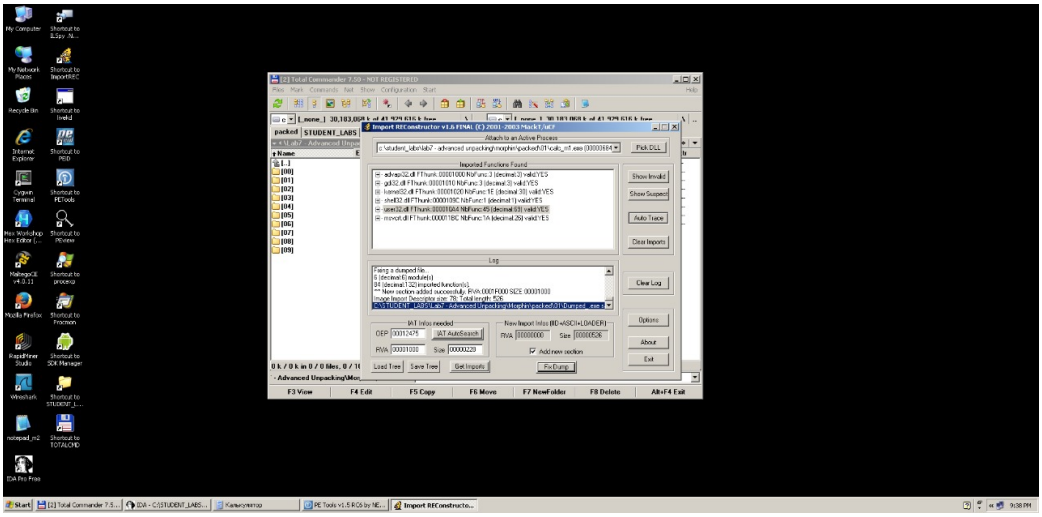

Figure 11: ImpREC detailed view and log for sample 01. "Exact call" notes are benign; final IAT is valid.

5.4 Fix Dump & Run

The repaired file `Dumped_.exe` launched and displayed the Calculator UI without errors.

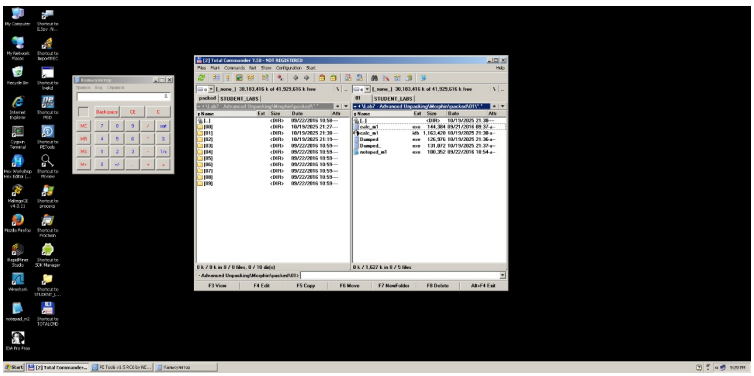

Figure 12: Fixed dump from sample 01 running correctly (Calculator UI visible).

Result (01). OEP captured, process dumped, IAT rebuilt, fixed dump executes correctly.

6 Summary and Conclusion

Both `calc_m0.exe` (00) and `calc_m1.exe` (01) were successfully unpacked using the same workflow:

1. Trace Morphine stub → reach OEP (Figs. 2, 8)
2. Dump at OEP with PETools
3. Rebuild IAT in ImpREC (Figs. 5, 4, 11, 10)
4. Fix dump and verify execution (Figs. 6, 12)
5. For both the Calc Files m0 and m1 the OEP was 00012475

Despite polymorphic differences, both samples decrypted in memory and transferred control into the original code via a computed jump. The resulting fixed binaries run as functional Calculator applications, meeting all assignment requirements.